Opérateurs et structures conditionnelles

420-249-AH Programmation mobile iOS

1. Pourquoi utiliser des opérateurs ?

Les opérateurs permettent de manipuler et comparer des valeurs.

Exemples :

```
let prix = 20
let quantite = 3

let total = prix * quantite
```

Résultat :

```
Console
60
```

Ils servent notamment à :

- Calculer
- Comparer
- Tester une condition
- Combiner plusieurs conditions

2. Les opérateurs arithmétiques

Swift utilise les opérateurs mathématiques classiques :

Opérateur	Signification
+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Reste d'une division

Exemple :

```
let a = 10
let b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
```

3. Addition et soustraction

```
let prix1 = 25
let prix2 = 15

let total = prix1 + prix2

print(total)
```

Résultat :

```
Console
40
```

Soustraction :

```
let solde = 100
let achat = 35

let nouveauSolde = solde - achat

print(nouveauSolde)
```

Résultat :

```
Console
65
```

4. Multiplication

```
let prix = 12
let quantite = 4

let total = prix * quantite

print(total)
```

Résultat :

```
Console
48
```

Exemple avec Double :

```
let prix = 12.50
let quantite = 4.0

let total = prix * quantite

print(total)
```

5. Attention à la division des entiers

Exemple :

```
let a = 10
let b = 3

let resultat = a / b

print(resultat)
```

Résultat :

```
Console
3
```

Pourquoi pas 3.3333 ? Parce que a et b sont des Int.

Avec des nombres décimaux :

```
let a = 10.0
let b = 3.0

let resultat = a / b

print(resultat)
```

On obtient une valeur décimale.

6. L'opérateur modulo %

L'opérateur % retourne le reste d'une division.

Exemple :

```
let resultat = 10 % 3

print(resultat)
```

Résultat :

```
Console
1
```

Autres exemples :

- 8 % 2 → 0
- 9 % 2 → 1
- 10 % 4 → 2

Utilisation fréquente : déterminer si un nombre est pair ou impair.

```
let nombre = 8

print(nombre % 2)
```

7. Modifier une variable

Supposons :

```
var compteur = 10
```

On peut écrire :

```
compteur = compteur + 1
```

Swift propose une écriture plus courte :

```
compteur += 1
```

Autres exemples :

```
compteur -= 1
compteur *= 2
compteur /= 2
```

8. Les opérateurs de comparaison

Les opérateurs de comparaison produisent une valeur de type :

```
Bool
```

Les principaux opérateurs sont :

Opérateur	Signification
==	égal à
!=	différent de
>	supérieur à
<	inférieur à
>=	supérieur ou égal à
<=	inférieur ou égal à

9. Comparer deux valeurs

```
let age = 20

print(age == 20)
print(age > 18)
print(age < 18)
```

Résultat :

```
Console
true
true
false
```

Les expressions suivantes retournent donc toujours true ou false.

10. = et ==

Attention à ne pas confondre :

```
=
```

avec :

```
==
```

= sert à affecter une valeur :

```
var age = 20

age = 21
```

== sert à comparer :

```
age == 21
```

On peut lire : age est-il égal à 21 ?

11. Prendre une décision avec if

Une application doit souvent prendre des décisions.

Exemple :

Si le mot de passe est correct → autoriser l'accès

Sinon → afficher une erreur

En Swift :

```
if condition {
    // instructions
}
```

12. Première condition

```
let age = 20

if age >= 18 {
    print("Accès autorisé")
}
```

La condition est :

```
age >= 18
```

Elle peut retourner true ou false. Si elle est vraie, le bloc est exécuté.

13. if et else

else permet de traiter le cas contraire.

```
let age = 16

if age >= 18 {
    print("Majeur")
} else {
    print("Mineur")
}
```

Résultat :

```
Console
Mineur
```

Logique :

```
age >= 18 ?
      |
  ┌───┴───┐
 Oui     Non
  |       |
Majeur  Mineur
```

14. Exemple — Réussite ou échec

```
let note = 72

if note >= 60 {
    print("Réussite")
} else {
    print("Échec")
}
```

Si :

```
note = 72
```

Résultat :

```
Console
Réussite
```

Si :

```
note = 45
```

Résultat :

```
Console
Échec
```

15. Plusieurs possibilités avec else if

Une décision peut comporter plusieurs cas.

```
let note = 82

if note >= 90 {
    print("Excellent")
} else if note >= 80 {
    print("Très bien")
} else if note >= 70 {
    print("Bien")
} else if note >= 60 {
    print("Réussite")
} else {
    print("Échec")
}
```

16. L'ordre des conditions est important

Considérons :

```
let note = 95
```

Avec :

```
if note >= 60 {
    print("Réussite")
} else if note >= 90 {
    print("Excellent")
}
```

Le résultat sera :

```
Console
Réussite
```

La deuxième condition ne sera jamais testée.

Il faut donc commencer par la condition la plus restrictive :

```swift
if note >= 90 {
    print("Excellent")
} else if note >= 60 {
    print("Réussite")
}
```

17. Les opérateurs logiques

Il est souvent nécessaire de tester plusieurs conditions en même temps.

Swift propose notamment :

Opérateur	Signification
&&	ET
||	OU
!	NON

18. L'opérateur &&

&& signifie ET. Les deux conditions doivent être vraies.

```swift
let age = 22
let possedePermis = true

if age >= 18 && possedePermis {
    print("Condition respectée")
}
```

Logique :

```
age >= 18
        ET
possedePermis == true
```

19. L'opérateur ||

|| signifie OU. Au moins une des conditions doit être vraie.

```
let estAdministrateur = false
let estResponsable = true

if estAdministrateur || estResponsable {
    print("Accès autorisé")
}
```

Ici, le résultat est :

```
Console
Accès autorisé
```

20. L'opérateur !

! signifie NON. Il inverse une valeur booléenne.

```
let estConnecte = false

if !estConnecte {
    print("Connexion requise")
}
```

On peut lire : si la personne n'est pas connectée.

21. Exemple avec plusieurs conditions

```
let age = 25
let estMembre = true

if age >= 18 && estMembre {
    print("Accès complet")
} else {
    print("Accès refusé")
}
```

Autre exemple :

```
let temperature = 28
let pluie = false

if temperature > 20 && !pluie {
    print("Bonne journée pour sortir")
}
```

22. Comparer des chaînes

Les chaînes peuvent également être comparées.

```swift
let motDePasse = "swift123"

if motDePasse == "swift123" {
    print("Mot de passe correct")
} else {
    print("Mot de passe incorrect")
}
```

On peut aussi utiliser :

```swift
!=
```

Exemple :

```swift
if motDePasse != "swift123" {
    print("Accès refusé")
}
```

23. Comparer des valeurs booléennes

Exemple :

```swift
let estConnecte = true

if estConnecte == true {
    print("Bienvenue")
}
```

Mais on peut écrire plus simplement :

```swift
if estConnecte {
    print("Bienvenue")
}
```

Et :

```swift
if !estConnecte {
    print("Connexion requise")
}
```

24. La structure switch

Lorsqu'une même valeur peut correspondre à plusieurs cas, switch peut rendre le code plus lisible.

Exemple :

```
let jour = 2

switch jour {
case 1:
    print("Lundi")

case 2:
    print("Mardi")

case 3:
    print("Mercredi")

default:
    print("Autre jour")
}
```

25. Le rôle de default

Un switch doit prévoir tous les cas possibles.

default: représente les valeurs qui n'ont pas été traitées précédemment.

Exemple :

```
let choix = 10

switch choix {
case 1:
    print("Ajouter")

case 2:
    print("Modifier")

case 3:
    print("Supprimer")

default:
    print("Choix invalide")
}
```

26. if ou switch ?

if est particulièrement pratique pour tester :

```
note >= 60
age < 18
prix > 100
```

Exemple :

```
if age >= 18 {
}
```

switch est pratique lorsque l'on compare une valeur à plusieurs possibilités :

```
switch choix {
case 1:
case 2:
case 3:
default:
}
```

Il n'existe pas une seule bonne solution dans tous les cas.

L'objectif est d'utiliser la structure qui rend le code le plus clair.

27. Exemple guidé — Calcul du prix d'un billet

Données :

```
let age = 17
let prixNormal = 20.0
```

Règles :

- Moins de 12 ans → 50 % du prix
- 12 à 17 ans → 75 % du prix
- 18 ans et plus → prix normal

Solution possible :

```
let age = 17
let prixNormal = 20.0
var prixFinal = prixNormal

if age < 12 {
    prixFinal = prixNormal * 0.5
} else if age < 18 {
    prixFinal = prixNormal * 0.75
} else {
    prixFinal = prixNormal
}

print("Prix final : \(prixFinal) $")
```

28. Exercice — Évaluation d'une note

Créer :

```
let note = 78
```

Afficher l'appréciation suivante :

- 90 à 100 → Excellent
- 80 à 89 → Très bien
- 70 à 79 → Bien
- 60 à 69 → Réussite
- 0 à 59 → Échec

Pour :

```
let note = 78
```

le programme doit afficher :

```
Console
Bien
```

Utiliser :

```
if
else if
else
```

29. Exercice — Validation d'un accès

Créer les données suivantes :

```
let age = 22
let estConnecte = true
let abonnementActif = true
```

Afficher :

```
Console
Accès autorisé
```

seulement lorsque : âge >= 18 ET connexion active ET abonnement actif.

Sinon :

```
Console
Accès refusé
```

30. Exercice — Nombre pair ou impair

Créer :

```
let nombre = 17
```

En utilisant %, déterminer si le nombre est pair ou impair.

Résultat attendu :

```
Console
17 est impair
```

Indice :

```
nombre % 2
```

Un nombre pair donne un reste de 0.

31. Exercice — Mini système de connexion

Créer :

```
let nomUtilisateur = "admin"
let motDePasse = "1234"
```

L'accès est autorisé uniquement lorsque les deux valeurs sont correctes.

Résultat possible :

```
Console
Connexion réussie
```

ou :

```
Console
Nom d'utilisateur ou mot de passe incorrect
```

Utiliser :

```
==
&&
if
else
```

32. À retenir

Opérateurs arithmétiques :

```
+   -   *   /   %
```

Opérateurs de comparaison :

```
==   !=   >   <   >=   <=
```

Opérateurs logiques :

```
&&   ||   !
```

Structures conditionnelles :

```
if
else if
else
switch
```

Une condition produit essentiellement une réponse : true ou false, et permet au programme de prendre une décision en fonction des données.